

答疑课堂（三） | 第三章Rust应用篇思考题答案

唐刚 · Rust 语言从入门到实战



你好，我是 Mike。

你真的很棒！已经学完我们这门 30 讲正文内容了，最后我们还是和前面两章一样处理一下第三章应用篇的思考题。这部分思考题动手操作的内容比较多，我希望你真的可以自己动手敲敲代码，在我给的示例里做出自己想要的效果。

话不多说，我们开始吧！

🔗 21 | Web 开发（上）：如何使用 Axum 框架进行 Web 后端开发？

思考题

请你说一说 Request/Response 模型是什么，Axum 框架和其他 gRPC 框架（比如 Tonic）有什么区别？

答案

Request/Response 模型是一种通用的网络模型架构，用于简化跨越网络的数据传输操作。在这种模型中，客户端发送一个请求（Request），服务器端接收并处理该请求，然后返回一个响应（Response）给客户端。这种模型广泛应用于各种网络应用和协议中，如 HTTP、FTP 等。Request / Response 模型就是一来一回交互。

Axum 框架和其他 gRPC 框架主要是通信模式不一样。Axum 提供了一种基于 actor 模型的通信模式。gRPC 模型在 Request / Response 模型基础上构造了更多的交互模型支持。

模型	特点
简单 RPC	传入一个 Request 对象，返回一个 Response 对象
服务端流式 RPC	传入一个 Request 对象，服务端可以返回多个 Response 对象
客户端流式 RPC	客户端传入多个 Request 对象，服务端返回一个 Response 对象
双向流式 RPC	传入多个 Request 对象，返回多个 Response 对象

🔗 22 | Web 开发（下）：如何实现一个 Todo List 应用？

思考题

当 Axum Handler 中有多个参数的时候，你可以试着改变一下参数的顺序，看看效果有没有变化。并在这个基础上说一说你对声明式参数概念的理解。

答案

声明式参数，简单来说就是参数的顺序不影响意图表达。因此在 Axum 中，解包器之间可以任意互换，这让 axum 的 handler 不那么在意参数顺序，而在意意图表达。这种设计有助于降低心智负担。

但是也要注意，对于要消耗 http request body 的解包器，如 Json、body 等，需要放在最后，原因也很简单，消耗掉了别人就用不了了。

🔗 23 | Rust 与大模型：用 Candle 做一个聊天机器人

思考题

你可以在我的示例上继续捣鼓，添加 GPU 的支持，在 Linux、Windows、macOS 多种平台上测试一下。

答案

GPU 用于模型推理加速。目前 candle quantized 模型的 GPU 支持还在 todo 清单之中，已经规划。这也是给 candle 做贡献的机会。safetensors 格式的模型 GPU 支持已经可用，可以看 candle 官方的示例，一般只需要在运行命令行的时候添加 `--features cuda` 就行，用 `--help` 查看。

🔗 24 | Rust 图像识别：利用 YOLOv8 识别对象

思考题

请你开启 cuda 或 metal 特性尝试一下，使用不同的预训练模型看一下效果差异。另外你还可以换用不同的图片来测试一下各种识别效果。

答案

开启 cuda 或 metal 特性只需要在命令行中加参数即可，具体可以 `--help` 看一下参数。

我们用的 M 模型做测试，你可以选择 S、L、X 等模型试试。S 速度最快，精度最低，模型体积最小。越往上走，模型速度变慢，精度变高，模型体积变大。实际场合中，特别是摄像头实时监控，往往用 S 最多，因为需要最快速检测。

🔗 25 | Rust GUI 编程：用 Slint 为 Chatbot 实现一个界面

思考题

这节课的示例非常原型化，代码还有很多可以改进的地方，请你思考一下并指出一两处。

答案

1. 消息窗口没有处理自动滚动到最下面的逻辑。

2. 消息窗口可以像微信那样左右对齐布局，机器人的消息靠左，自己的消息靠右。

🔗 26 | Rust GUI 编程：用 Slint 为 YOLOv8 实现一个界面

思考题

这节课的代码实现有一个性能上的问题，就是每次点击 Detect Objects 或 Detect Poses 的时候，实际上都重复加载了模型，你想想如何优化这个点？

答案

可以把加载模型的逻辑放在界面启动完成后，通过回调函数加载。因为 yolo 模型还是比较小，因此可以直接写在 GUI 界面的回调中加载。

当点击识别对象和识别姿势按钮时，直接调用加载后的模型执行推理。这样反应速度能加快不少。

🔗 27 | Rust Bevy 游戏开发：用 300 行代码做一个贪吃蛇游戏

思考题

这节课的代码还有个问题，就是食物有可能在已经产生过的地方产生，也有可能在蛇身上产生，请问如何处理这个 Bug？

答案

添加一个 Resource，跟踪维护食物的 Positions，随机产生食物的时候，需要遍历判断是否在已有的食物的 Positions 和蛇的 Positions 上。如果有就重新计算随机数，再重复这个过程。

食物的 Positions 可以用 HashSet 这种数据结构管理。需要注意的是，当食物被吃掉后，也需要在这个结构中对应的清理。

🔗 28 | Nom：用 Rust 写一个 Parser 解析器

思考题

请尝试用 Nom 解析一个简单版本的 CSV 格式文件。

答案

首先，你需要添加 Nom 依赖：

```
1 cargo add nom
```

[复制代码](#)

然后，你可以使用以下代码来解析 CSV 文件（一个简单示例）：

```
1 use nom::{
2     bytes::complete::tag,
3     character::complete::{alphanumeric1 as field, line_ending},
4     multi::separated_list1,
5     sequence::separated_pair,
6     IResult,
7 };
8
9 fn csv_line(input: &str) -> IResult<&str, Vec<&str>> {
10     separated_list1(tag(","), field)(input)
11 }
12
13 fn csv(input: &str) -> IResult<&str, Vec<Vec<&str>>> {
14     separated_list1(line_ending, csv_line)(input)
15 }
16
17 fn main() {
18     let input = "field1,field2,field3\nfield1,field2,field3";
19     let result = csv(input);
20     match result {
21         Ok( (_, parsed )) => {
22             for line in parsed {
23                 println!("{:?}", line);
24             }
25         }
26         Err(e) => println!("Error: {:?}", e),
27     }
28 }
```

[复制代码](#)

🔗29 | Unsafe 编程（上）：Unsafe Rust 中那些被封印的能力

思考题

Unsafe Rust 比 C 语言更安全吗，为什么？

答案

是的，总体来看 Unsafe Rust 比 C 语言更安全。虽然是 Unsafe，可以用指针。但是，Unsafe 依然属于所有权这样的语法范畴。

来源：十八哥

🔗30 | Unsafe 编程（下）：使用 Rust 为 Python 写一个扩展

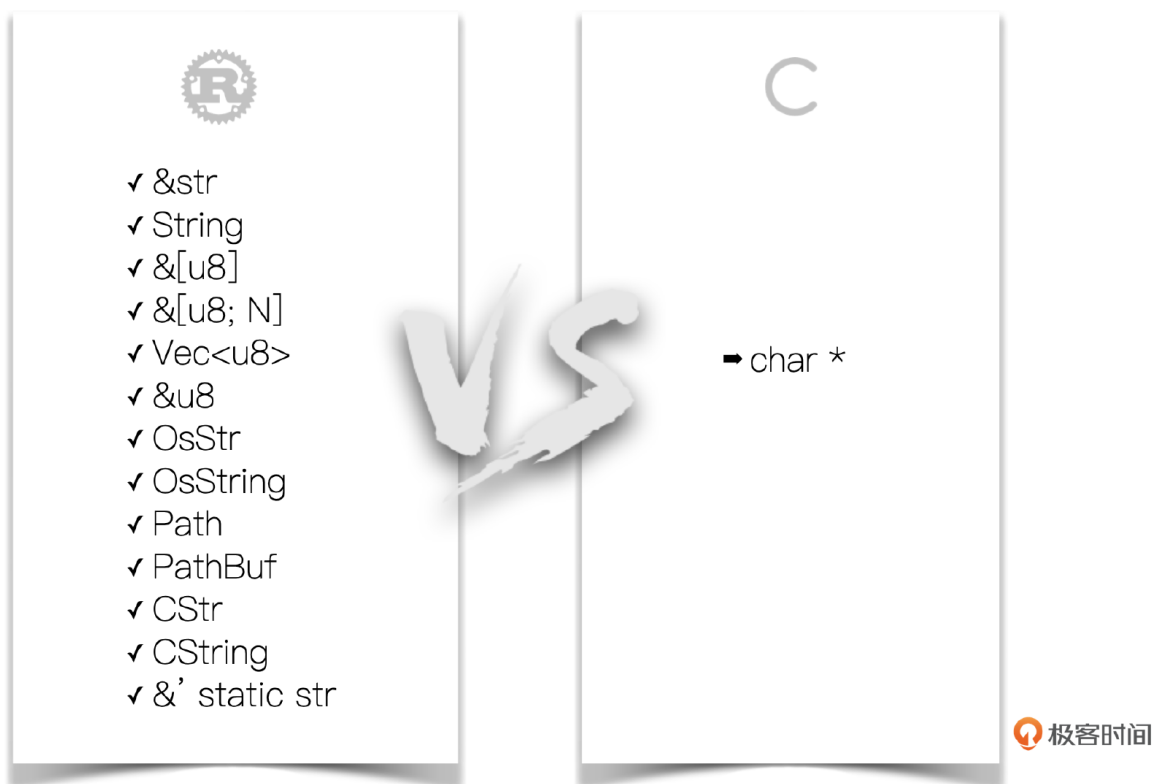
思考题

1. C 语言中的 char 与 Rust 中的 char 的区别在哪里？
2. C 语言的字符串与 Rust 中的 String 区别在哪里？
3. 如何将 C 语言的字符串类型映射到 Rust 的类型中来？

答案

C 语言中的 char 就是一个字节，Rust 中的 char 是 4 个字节，存的是字符的 unicode scalar 值。

❗ Rust 与 C 语言字符串对比



C 中的字符串统一叫做 `char *`，这确实很简洁，相当于是统一的抽象。但是这个统一的抽象也付出了代价，就是丢失了很多额外的信息。Rust 把字符串在各种场景下的使用给模型化、抽象化了。相比 C 语言的 `char *`，多了建模的过程，在这个模型里面多了很多额外的信息。

C 语言的字符串与 Rust 中的 `String` 区别主要在于以下几个方面：

- 存储方式**：在 C 语言中，字符串通常以字符数组的形式存储，通过以 null 字符（`'\0'`）作为字符串的结束标记。而在 Rust 中，`String` 是动态分配的，基于 UTF-8 编码，并使用指针结构来追踪其起始和结束位置。
- 内存管理**：在 C 语言中，字符串的内存管理需要手动处理，包括分配、释放等。而在 Rust 中，`String` 的内存管理由 Rust 的所有权机制自动管理，无需手动释放。
- 安全性**：C 语言由于其原始的内存管理方式，容易导致内存泄漏、缓冲区溢出等问题。而 Rust 的类型系统和内存管理机制提供了更强的安全性保障，杜绝了这类问题的发生。

由于这些不同，C 语言的字符串类型在 Rust 中映射为 [🔗 CStr](#)。

全部留言 (1)

最新 精选



一带一路

2024-01-03 来自四川

结课了嘛？

编辑回复：对，马上结课啦



1